Kubernetes On-Premises: Architecture and Setup Guide

Author: Sachin Dubey | First published: Oct 16, 2026 | Revision 4: Nov 2, 2026

On-prem Kubernetes runs on servers you own: bare metal or virtual machines (VMs) in your data center, with containers running your apps on top. This guide explains the architecture, virtualization versus containerization, and a step-by-step kubeadm setup with every command.

Revision 2 added an AWS EC2 section, a pre-flight checklist, a check after every step, and a problems and fixes chapter built from real errors hit while building a cluster.

Revision 3 added Problem 14 (Calico stuck because the pod CIDR in custom-resources.yaml does not match kubeadm init), a mandatory CIDR check in Step 8b, and corrected the earlier statement that 192.168.0.0/16 is the default in the Calico download. The file downloaded for Calico v3.32.2 contains 10.244.0.0/16.

Revision 4 adds everything learned while building the cluster on VirtualBox VMs (Ubuntu Desktop on a Windows laptop): a new Section 5.2 (VirtualBox networking: unique IPs with a Host-only adapter and static IPs with nmcli), VirtualBox notes inside Steps 1, 2, 6, 8, 10 and 11, a stronger permanent swap fix, and Problems 15 to 25.

How to use this guide

Every step is a short block of commands. Run the blocks in order, and run each Check before moving to the next step. If a check fails, stop and see Section 9 instead of continuing.

Rules that prevent most errors:

1. Copy commands from the .md file or GitHub, not from the PDF. PDF viewers can insert line breaks into long commands when you copy them. A broken line was the cause of the "NO_PUBKEY" error in Problem 9.
2. Use the same Kubernetes version on every node. Pick one version and use it everywhere.
3. Use private IPs in /etc/hosts, kubeadm init and kubeadm join. Replace every example IP in this guide with your real one (hostname -I). On VirtualBox, never use 10.0.2.15 (see Section 5.2).

4. Follow the order: Part A on all nodes, then Part B on the master, then Part C on the workers.
5. Run the check after each step. Do not move on until it passes.
6. Run the commands one block at a time, not the whole guide at once. Run create, apply and watch commands as separate commands, never pasted together on one line (see Problem 14 and Problem 23).
7. The pod CIDR must be identical in two places: the --pod-network-cidr of kubeadm init (Step 6) and the cidr: line of Calico's custom-resources.yaml (Step 8b). Check it with grep cidr custom-resources.yaml before you install Calico. The pod CIDR must also not overlap your node network.
8. Swap must be off permanently. Verify with free -h after every reboot (Problem 16).
9. The master never joins anything, and Calico is installed once from the master. Only workers run kubeadm join.

Table of Contents

1. What does On-Prem mean?
2. On-Prem Architecture
3. Virtualization vs Containerization
4. Example Cluster Used in This Guide
5. Running on Cloud VMs and VirtualBox (5.1 AWS EC2 notes, 5.2 VirtualBox notes)
6. Pre-flight Checklist
7. Step-by-Step Setup with kubeadm
8. On-Prem Extras You Must Plan
9. Common Problems and Fixes
10. Starting Over (Reset and Uninstall)
11. Command Cheat Sheet

1. What does On-Prem mean?

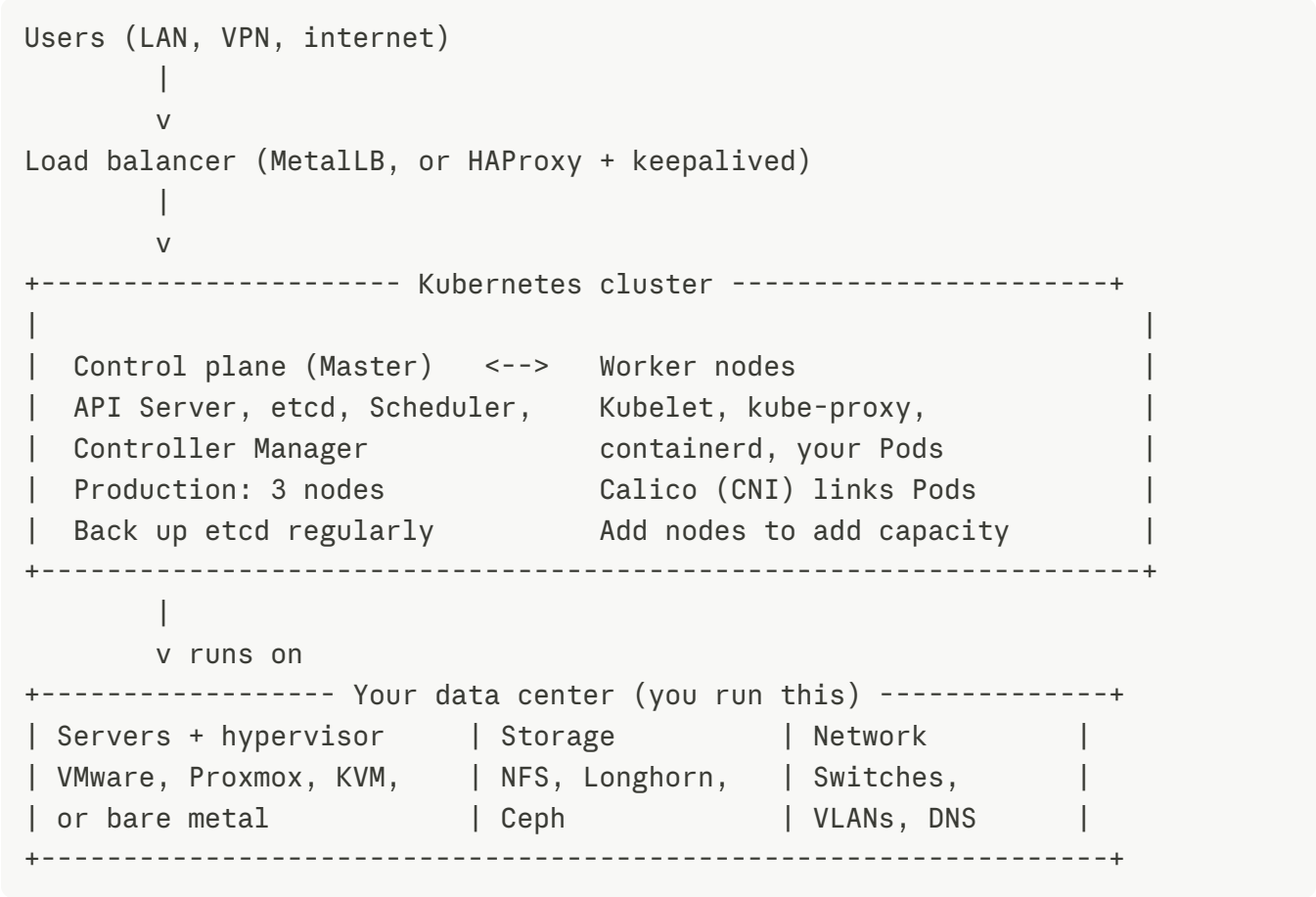
On-premises (on-prem) means the servers, network and storage are in your own office or data center, not rented from a cloud such as AWS, Azure or GCP.

The Kubernetes parts are the same as in the cloud (API Server, etcd, Scheduler, Controller Manager, Kubelet, kube-proxy). What changes is that you must provide things the cloud gives for free:

- The machines (physical servers or VMs)
- A load balancer for users and for the API Server
- Persistent storage for data
- Networking, firewall and DNS

2. On-Prem Architecture

The cluster sits on your own servers. Users enter through a load balancer, the control plane makes decisions, worker nodes run the Pods, and your own storage and network sit underneath.



Layers explained

Layer	What it is	On-prem choice
Users	People or apps that call your service	Office network, VPN or internet
Load balancer	Spreads traffic across Nodes and gives a stable IP	MetalLB, or HAProxy plus keepalived
Control plane (Master)	API Server, etcd, Scheduler, Controller Manager	1 node for a lab, 3 nodes for production
Worker nodes	Run your Pods with Kubelet, kube-proxy and containerd	VMs or bare metal servers
Pod network (CNI)	Lets Pods on different Nodes talk	Calico, Flannel or Weave Net
Storage	Keeps data when Pods restart	NFS, Longhorn, Ceph
Physical layer	Servers, switches, power, hypervisor	VMware, Proxmox, KVM, VirtualBox (labs), or bare metal

3. Virtualization vs Containerization

These are two different ways to share one physical server between many workloads.

Virtualization uses a hypervisor to split one server into several virtual machines. Each VM has its own full operating system, so it is heavy (GBs of disk, minutes to boot) but strongly isolated. Examples: VMware vSphere, Proxmox, KVM, Hyper-V, VirtualBox.

Containerization packages an app with its libraries and runs it on the host's shared OS kernel. Containers are light (MBs, start in seconds) but share the kernel. Examples: Docker, containerd, CRI-O.

```
Virtualization          Containerization        Kubernetes on-prem
--------------          ----------------        ------------------
App A | App B           App A | App B           Pods (your apps)
Guest OS | Guest OS     Container runtime       Node OS + containerd
Hypervisor              Host OS (shared)        Hypervisor, or skip it
Server hardware         Server hardware         Server hardware
```

On-prem Kubernetes stacks both: an optional hypervisor underneath, containers on top.

Comparison

Point	Virtualization (VM)	Containerization (Container)
Isolation	Own OS and kernel per VM	Shared host kernel
Size	GBs	MBs
Startup time	Minutes	Seconds
Performance	Some overhead from hypervisor and guest OS	Near native
Density per server	Tens of VMs	Hundreds of containers
Managed by	Hypervisor (vCenter, Proxmox)	Kubernetes
Best for	Running full OS, isolating teams, hosting the Nodes	Packaging and scaling apps

Which one do we use in an On-Prem setup?

Both, at different layers.

- **Containerization is always used.** Kubernetes exists to run containers, so your apps run as containers (via containerd) inside Pods.
- **Virtualization is optional and sits underneath.** The Nodes themselves can be VMs (most common, because they are easy to create, snapshot and resize) or bare metal servers (best raw performance, used for large or latency-sensitive clusters).

Simple rule: for labs, learning and small or medium companies, use VMs on VMware, Proxmox, KVM or VirtualBox as your Nodes. For heavy workloads such as databases, AI or big data, consider bare metal Nodes. In both cases the apps are containers.

4. Example Cluster Used in This Guide

This guide uses three Ubuntu machines (VMs or physical). It was written for Ubuntu 22.04 and 24.04, and later tests ran on newer Ubuntu releases (including Ubuntu Desktop on VirtualBox). Change the IP addresses to match your network.

Hostname	Role	IP address (example)	CPU	RAM
k8s-master	Control plane	192.168.1.10	2 or more	2 GB or more (4 GB better)
k8s-worker1	Worker	192.168.1.11	2	2 GB or more
k8s-worker2	Worker	192.168.1.12	2	2 GB or more

Requirements: static IPs, full network connectivity between the machines, unique hostnames, internet access to download packages, and a user with sudo rights.

Important: the 192.168.1.x addresses are only examples. Everywhere this guide shows one, replace it with the real private IP of your machine. Find it with hostname -I (on VirtualBox, hostname -I lists the NAT address 10.0.2.15 first, so use ip a show enp0s8 instead, see Section 5.2).

Example values used for the VirtualBox lab in Section 5.2:

Hostname	Host-only IP	Pod CIDR
k8s-master	192.168.56.10	10.244.0.0/16
k8s-worker1	192.168.56.11	10.244.0.0/16
k8s-worker2	192.168.56.12	10.244.0.0/16

5. Running on Cloud VMs and VirtualBox

5.1 AWS EC2 notes

If you build this lab on AWS EC2 (or any cloud) instead of your own servers, these points caused real failures:

1. **Use private IPs, not public IPs.** In /etc/hosts, in --apiserver-advertise-address, and in kubeadm join, always use the private IP (for example 172.31.x.x). Public IPs change when an instance is stopped and started, cost data transfer, and are not assigned to the network interface, so the API server cannot bind to them.
2. **Find the real IP first.** The example 192.168.1.10 does not exist on an EC2 instance. Running kubeadm init with a non-existent IP makes the API server unreachable and init fails (Problem 3).
3. **Pod CIDR must not overlap your VPC.** The default AWS VPC uses 172.31.0.0/16, so the pod range 192.168.0.0/16 used in this guide is safe. If your VPC uses 192.168.x.x, pick another pod range (for example 10.244.0.0/16) and use it everywhere (Step 6 and Step 8b).
4. **Instance size.** The control plane needs at least 2 vCPU and about 2 GB RAM. Use t3.medium (2 vCPU, 4 GB) or larger. t2.micro and t3.micro will fail.
5. **Security group.** The simplest working setup is one rule that allows all traffic from the same security group (so nodes can talk to each other), plus SSH from your IP. Put all three instances in the same security group. See the port table in Section 8 for the strict version.
6. **Name your machines.** EC2 gives names like ip-172-31-8-191. Rename each node (Step 1) so names match /etc/hosts and are unique.
7. **Testing a NodePort app.** Open the NodePort range (30000-32767) to your own IP in the security group, then browse to http://<worker-public-ip>:<nodeport>.
8. **MetalLB does not work on AWS.** Layer 2 mode needs ARP, which AWS VPCs do not support. Use the AWS Load Balancer Controller or a NodePort instead. MetalLB is for real on-prem networks.

5.2 VirtualBox notes (local VMs on a laptop)

This section comes from a real build on VirtualBox with three Ubuntu Desktop VMs (master-node, worker-node-1, worker-node-2). Do it before Part A.

5.2.1 The key rule: every VM needs its own unique IP

A new VirtualBox VM has one **NAT** adapter. In NAT mode every VM gets the same address (10.0.2.15) on its own private network, so the VMs cannot see each other and Kubernetes cannot work. Never use 10.0.2.15 in /etc/hosts, kubeadm init or kubeadm join. On a worker, 10.0.2.15 is the worker itself, so a join pointed at it is refused (Problem 17).

The fix is a second adapter on a shared **Host-only** network:

Adapter	Type	Interface	Purpose
Adapter 1	NAT	enp0s3	Internet access (apt, image pulls). Same 10.0.2.15 on every VM; ignore it for Kubernetes.
Adapter 2	Host-only	enp0s8	Node-to-node traffic and the Kubernetes IP.

5.2.2 Add the Host-only adapter (all 3 VMs)

1. Shut down the VM.
2. VirtualBox > select the VM > Settings > **Network** (left sidebar, not System) > **Adapter 2** tab.
3. Tick **Enable Network Adapter**, Attached to: **Host-only Adapter**, Name: the same Host-only network on all three VMs.
4. Leave Adapter 1 as NAT. Click OK.
5. If the Name list is empty: File > Tools > Network Manager > Host-only Networks > Create, then reopen Settings.

Check (after boot, on every VM):

```
ip a
```

enp0s8 must be UP and show an address like 192.168.56.x.

5.2.3 Set a static IP (Ubuntu Desktop uses NetworkManager)

The Host-only adapter gets a **dynamic DHCP address** with a short lease (for example 192.168.56.102), which can change. Kubernetes needs fixed IPs.

On Ubuntu Desktop, /etc/netplan/ holds only 01-network-manager-all.yaml, which hands control to NetworkManager. Do not put the static IP in that file. Use nmcli. First find the connection whose DEVICE is enp0s8 (usually "Wired connection 2"):

```
nmcli -p connection show | cat
```

Then, on each node with its own address:

```
sudo nmcli connection modify "Wired connection 2" ipv4.method manual
ipv4.addresses 192.168.56.10/24
sudo nmcli connection up "Wired connection 2"
ip a show enp0s8
```

Node	Static IP
k8s-master	192.168.56.10/24
k8s-worker1	192.168.56.11/24
k8s-worker2	192.168.56.12/24

Do not set a gateway on this adapter; internet continues through the NAT adapter. The default Host-only DHCP range starts at .101, so .10 to .12 do not collide.

Check: ip a show enp0s8 shows the static address without the word dynamic. Then from every node:

```
ping -c 3 192.168.56.10
ping -c 3 192.168.56.11
ping -c 3 192.168.56.12
```

All pings must succeed before you continue. If one fails, the VMs are not on the same Host-only network (re-check Adapter 2 on both VMs).

5.2.4 Where VirtualBox differs from the rest of the guide

Step	On VirtualBox do this
Step 1 (/etc/hosts)	Use the 192.168.56.x addresses. Remove old lines first, because tee -a adds duplicates.
Step 2 (swap)	Ubuntu Desktop uses /swapfile and turns it back on at boot. Verify with free -h after every reboot (Problem 16).
Step 6 (init)	Do not use hostname -I to find MASTER_IP: it returns 10.0.2.15 first. Set MASTER_IP=192.168.56.10 by hand. Use pod CIDR 10.244.0.0/16, because 192.168.0.0/16 overlaps 192.168.56.0/24 (Problem 4).
Before init and join	Set --node-ip in /etc/default/kubelet on every node so kubelet does not pick the NAT address.
Step 8 (Calico)	custom-resources.yaml already contains 10.244.0.0/16, so no sed edit is needed. Tell Calico to use enp0s8.

Step	On VirtualBox do this
Step 10 (join)	Run swap off, kubeadm reset -f and set --node-ip on the worker first. The join command must start with 192.168.56.10:6443.
After join	Workers show NotReady until their calico-node pod runs. Wait 3 to 5 minutes (Problem 25).

5.2.5 Sizing and cloned VMs

- Master: 2 CPU and 4 GB RAM. Workers: 2 CPU and 2 GB or more.
- If VMs were cloned, each must have a unique hostname, MAC address, machine-id and product_uuid. Check with cat /etc/machine-id and sudo cat /sys/class/dmi/id/product_uuid.

6. Pre-flight Checklist

Complete this before you start. Fill in the table with your own values and keep it open while you work.

Item	Master	Worker 1	Worker 2
Hostname	k8s-master	k8s-worker1	k8s-worker2
Private IP (hostname -I, or ip a show enp0s8 on VirtualBox)	(fill in)	(fill in)	(fill in)
CPU / RAM	2+ / 2 GB+	2+ / 2 GB+	2+ / 2 GB+

- Three Ubuntu machines, each with at least 2 CPU and 2 GB RAM (4 GB better on the master)
- You can SSH into each machine and use sudo
- Each machine has internet access (curl -I https://pkgs.k8s.io returns a response)
- The machines can reach each other (same security group on AWS, same Host-only network on VirtualBox, or firewall ports open on-prem)
- Hostnames are unique

- You wrote down the three private IPs above (on VirtualBox: 192.168.56.10, .11 and .12 set statically, with all pings working, never 10.0.2.15)
- You chose one Kubernetes version for all nodes
- You chose one pod CIDR and wrote it down (192.168.0.0/16 on AWS or a LAN that does not use 192.168.x.x; 10.244.0.0/16 on VirtualBox)
- Swap is off and will stay off after a reboot

Choosing the Kubernetes version

Check https://kubernetes.io/releases/ for the supported versions. As of Oct 2026, v1.34 reaches end of life on Oct 27, 2026, while v1.35 and v1.36 are supported and v1.37 is the newest release. This guide uses v1.35 as the default (the VirtualBox lab ran v1.35.10).

- Building a new cluster: keep v1.35 (or choose another supported version) and use it on every node.
- Already installed v1.34 on some nodes (for example your master)? Use v1.34 on the remaining nodes too. Mixing versions causes confusing errors. You can upgrade the whole cluster later.
- Before choosing a version, also check that your Calico version supports it (see Step 8).

7. Step-by-Step Setup with kubeadm

kubeadm is the official tool to build a cluster. Part A runs on ALL nodes, then Part B runs on the master only, then Part C runs on the workers, then Part D verifies everything.

On VirtualBox, finish Section 5.2 first (Host-only adapter, static IPs, all pings working).

Part A: Prepare every node (master and workers)

Step 1. Set a hostname and name resolution

Run the matching hostname command on each machine. Then add the host entries on all machines, using your real private IPs from the checklist.

```
# On the master
sudo hostnamectl set-hostname k8s-master
# On worker 1
sudo hostnamectl set-hostname k8s-worker1
# On worker 2
sudo hostnamectl set-hostname k8s-worker2
```

On all nodes (replace the three IPs first):

```
printf '%s\n' '192.168.1.10 k8s-master' '192.168.1.11 k8s-worker1'
'192.168.1.12 k8s-worker2' | sudo tee -a /etc/hosts
```

On VirtualBox use the Host-only IPs. The same thing as a heredoc:

```
sudo tee -a /etc/hosts <<EOF
192.168.56.10 k8s-master
192.168.56.11 k8s-worker1
192.168.56.12 k8s-worker2
EOF
```

tee -a appends, so running it twice creates duplicate lines. If you already added wrong or duplicate IPs, edit the file on every node and fix or delete the bad lines:

```
sudo nano /etc/hosts
```

Check:

```
hostname
cat /etc/hosts
```

Do Steps 2 to 5 on every node, one node at a time (finish a node completely, then go to the next), or open one terminal per node.

Step 2. Turn off swap (permanently)

Kubelet refuses to run while swap is on. If swap comes back after a reboot, kubelet crashes in a loop and the API server never starts (Problem 16).

```
sudo swapoff -a
sudo sed -i '/swap/ s/^\([^#]\)/#\1/' /etc/fstab
```

Check:

```
free -h
grep swap /etc/fstab
```

The Swap row must show 0B and the fstab line must start with #. Ubuntu Desktop uses /swapfile, which is enabled from /etc/fstab at every boot, so the fstab line is the permanent part. Run free -h again after your first reboot.

Step 3. Load kernel modules and network settings

These let containers use the bridge network and let the node forward packets. Run these one line at a time (each is a single line, so nothing can break when pasted):

```
printf '%s\n' 'overlay' 'br_netfilter' | sudo tee /etc/modules-
load.d/k8s.conf
sudo modprobe overlay
sudo modprobe br_netfilter
printf '%s\n' 'net.bridge.bridge-nf-call-iptables = 1' 'net.bridge.bridge-nf-
call-ip6tables = 1' 'net.ipv4.ip_forward = 1' | sudo tee
/etc/sysctl.d/k8s.conf
sudo sysctl --system
```

Each tee command prints back the lines it wrote. Check that they look exactly like the text inside the quotes.

What each part does:

Item	Purpose
overlay module	OverlayFS, used by containerd to stack image layers
br_netfilter module	Makes bridged traffic pass through iptables (needed by kube-proxy and the CNI)
net.bridge.bridge-nf-call-iptables = 1	Bridged IPv4 traffic goes through iptables rules
net.bridge.bridge-nf-call-ip6tables = 1	Same, for IPv6
net.ipv4.ip_forward = 1	Lets the node forward packets between interfaces (Pod-to-Pod and Pod-to-Service traffic)

Check:

```
lsmod | grep -E "overlay|br_netfilter"
sysctl net.bridge.bridge-nf-call-iptables net.bridge.bridge-nf-call-ip6tables
net.ipv4.ip_forward
cat /proc/sys/net/ipv4/ip_forward
```

Both modules must be listed. All three sysctl values must show = 1, and the last command must print 1. Also check that sudo sysctl --system printed no error such as Path is not under /proc/sys/. If it did, see Problem 13.

Step 4. Install the container runtime (containerd)

This is the engine that actually runs containers.

```
sudo apt-get update
sudo apt-get install -y containerd
sudo mkdir -p /etc/containerd
containerd config default | sudo tee /etc/containerd/config.toml
# Use systemd cgroups (required by kubelet)
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/'
/etc/containerd/config.toml
sudo systemctl restart containerd
sudo systemctl enable containerd
```

Check:

```
grep SystemdCgroup /etc/containerd/config.toml
systemctl status containerd --no-pager
```

The first must show SystemdCgroup = true. The second must show active (running). Use --no-pager, or press q to leave the status screen (Problem 22).

Step 5. Install kubeadm, kubelet and kubectl

The commands are split into small pieces on purpose. Run them in order, and stop and fix if a check fails. Use the same version on every node.

5a. Set the version and prepare apt:

```
KUBE_VERSION=v1.35
KEYRING=/etc/apt/keyrings/kubernetes-apt-keyring.gpg
REPO=https://pkgs.k8s.io/core:/stable:/$KUBE_VERSION/deb
sudo apt-get update
sudo apt-get install -y apt-transport-https ca-certificates curl gpg
sudo mkdir -p -m 755 /etc/apt/keyrings
sudo rm -f $KEYRING
```

5b. Download the signing key, then convert it (two separate commands, no pipe):

```
curl -fsSL $REPO/Release.key -o /tmp/k8s-release.key
sudo gpg --dearmor -o $KEYRING /tmp/k8s-release.key
sudo chmod 644 $KEYRING
ls -l $KEYRING
```

Check: the ls line must show a file size greater than 0 (about 1 to 3 KB). If it says No such file or 0, stop and see Problem 9.

5c. Add the repository:

```
echo "deb [signed-by=$KEYRING] $REPO/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
cat /etc/apt/sources.list.d/kubernetes.list
```

Check: the cat output must be one line that contains your version (for example v1.35) and no $ characters.

5d. Install and pin the packages:

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

Check: during apt-get update you must see a pkgs.k8s.io line starting with Get: or Hit:. If you see Err: or NO_PUBKEY, go back to 5b.

5e. VirtualBox only: tell kubelet which IP to use. Without this, kubelet can register the NAT address 10.0.2.15. Use .10 on the master, .11 on worker1 and .12 on worker2:

```
echo 'KUBELET_EXTRA_ARGS=--node-ip=192.168.56.10' | sudo tee
/etc/default/kubelet
sudo systemctl restart kubelet
```

Part A final check (run on every node):

```
kubeadm version -o short
free -h
grep swap /etc/fstab
lsmod | grep -E "overlay|br_netfilter"
cat /proc/sys/net/ipv4/ip_forward
grep SystemdCgroup /etc/containerd/config.toml
systemctl is-active containerd
hostname
```

Every line must be right before you continue:

Command	Must show
kubeadm version -o short	A version such as v1.35.x, the same on every node
free -h	Swap: line with 0B
grep swap /etc/fstab	The swap line starts with #
lsmod …	Both overlay and br_netfilter listed
cat /proc/sys/net/ipv4/ip_forward	1
grep SystemdCgroup …	SystemdCgroup = true
systemctl is-active containerd	active
hostname	k8s-master, k8s-worker1 or k8s-worker2 (not ip-172-… and not the VM default name)

Notes:

- apt-mark hold stops a routine apt upgrade from changing your Kubernetes version by accident.
- Until you run kubeadm init or kubeadm join, kubelet restarts in a loop (activating (auto-restart)). This is normal, not an error. If it is still looping after init or join, check swap first (Problem 16).
- If you open a new terminal, the variables from 5a are gone. Run 5a again, or type the values directly.
- Ubuntu Desktop minimal installs have no vim. Use nano, or install vim (Problem 21).

Part B: Create the control plane (master only)

Step 6. Initialize the cluster

First find the master's real private IP and confirm it looks right. On a normal network or AWS:

```
hostname -I
MASTER_IP=$(hostname -I | awk '{print $1}')
echo $MASTER_IP
```

The printed address must be the private IP you wrote in the checklist (on AWS it looks like 172.31.x.x or 10.x.x.x).

VirtualBox: do not use that command. hostname -I lists the NAT address 10.0.2.15 first, so MASTER_IP would be wrong and the whole cluster would be bound to an address the workers cannot reach (Problem 18). Set it by hand and check the interface:

```
MASTER_IP=192.168.56.10
echo $MASTER_IP
ip a show enp0s8
```

Then initialize. On AWS or a LAN that does not use 192.168.x.x this guide uses the pod CIDR 192.168.0.0/16. On VirtualBox use 10.244.0.0/16. Remember the value you choose: Step 8b must use exactly the same one.

```
sudo kubeadm init --apiserver-advertise-address=$MASTER_IP --pod-network-
cidr=192.168.0.0/16
```

VirtualBox version:

```
sudo kubeadm init --apiserver-advertise-address=$MASTER_IP --pod-network-
cidr=10.244.0.0/16
```

This takes 1 to 3 minutes. When it succeeds, the last lines look like this (your IP, token and hash will differ):

```
Your Kubernetes control-plane has initialized successfully!
...
You should now deploy a pod network to the cluster.
...
kubeadm join 172.31.2.148:6443 --token <token> \
    --discovery-token-ca-cert-hash sha256:<hash>
```

Do these three things right away:

1. Do not run kubectl yet. If you do, it fails with The connection to the server localhost:8080 was refused, because your user has no kubeconfig. This is not a cluster failure. Go straight to Step 7.
2. Note the IP in the kubeadm join line. It is the master's private IP (on VirtualBox it must be 192.168.56.10; if it shows 10.0.2.15, reset and redo the init, Problem 18). Every node's /etc/hosts must use the same IP for k8s-master (see "Keep /etc/hosts in sync" below Step 7).

3. Treat the token and hash as secrets. Do not post screenshots or paste them in public places. They expire after 24 hours, and you can always create a new join command with Step 9. The printed join command says "as root", so on Ubuntu run it with sudo in front.

Optional: reach the API server from outside the VPC (for example kubectl from your laptop). Add the master's public IP to the API server certificate:

```
sudo kubeadm init --apiserver-advertise-address=$MASTER_IP --pod-network-
cidr=192.168.0.0/16 --apiserver-cert-extra-sans=<MASTER_PUBLIC_IP>
```

If kubeadm init fails or times out, do not re-run it on top of the failed attempt. Reset first, then check the causes in Problem 3:

```
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d $HOME/.kube
sudo systemctl restart containerd kubelet
```

Then run the init command again.

Step 7. Let your user run kubectl

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

The master shows NotReady until a network plugin is installed. This is expected.

If kubectl get nodes prints localhost:8080 ... connection refused instead, the three mkdir / cp / chown lines did not run correctly. Run them again (Problem 8). If it prints a connection refused for 10.0.2.15:6443 (or any other node IP) after a reboot, kubelet is down, usually because swap is on again (Problem 16).

Keep /etc/hosts in sync. The master's private IP is the address in the kubeadm join line (also shown by hostname -I on the master, or ip a show enp0s8 on VirtualBox). Make sure every node has this exact line, with that IP:

```
hostname -I
cat /etc/hosts
```

```
<MASTER_PRIVATE_IP> k8s-master
```

If you rebuilt or replaced the master instance, its private IP is different from before, and old /etc/hosts entries on the workers are now wrong. Fix them with sudo nano /etc/hosts on each node before joining.

Step 8. Install the Pod network (Calico v3.32.2)

Check https://docs.tigera.io/calico/latest/getting-started/kubernetes/quickstart for the current Calico version and its supported Kubernetes versions. Replace the version number in the URLs below if a newer one exists.

Rules for this step:

- Install Calico only from the master. Never install it by hand on workers: Kubernetes runs a calico-node pod on every node automatically.
- Use kubectl create, not kubectl apply. The CRDs are large and apply can fail with an "annotation too long" error.
- Install only one network plugin. If you applied Flannel earlier, remove it first: kubectl delete -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
- The pod CIDR in Calico must equal the --pod-network-cidr used in Step 6. The custom-resources.yaml you download in 8b is not guaranteed to contain your range. The v3.32.2 file contains 10.244.0.0/16, so with 192.168.0.0/16 you must change it (Step 8b); with 10.244.0.0/16 (VirtualBox) you change nothing. Skipping this check is the most likely reason Calico never starts (Problem 14).
- Run each command separately. Do not paste kubectl create ... and watch ... together, because the shell would treat watch as part of the first command.

8a. Install the CRDs and the Tigera Operator:

```
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.32.2/manifests/v1_c
rd_projectcalico_org.yaml
kubectl create -f
https://raw.githubusercontent.com/projectcalico/calico/v3.32.2/manifests/tige
ra-operator.yaml
```

Check: both commands print lines ending in created, and this shows the operator Running after a minute:

```
kubectl get pods -n tigera-operator
```

8b. Download the custom resources file and set the CIDR (mandatory check):

```
curl -O
https://raw.githubusercontent.com/projectcalico/calico/v3.32.2/manifests/cust
om-resources.yaml
grep -n cidr custom-resources.yaml
```

The grep line must show exactly the CIDR you used in Step 6. If it shows a different range (for example cidr: 10.244.0.0/16 while you used 192.168.0.0/16), change it now, before installing:

```
sed -i 's#10.244.0.0/16#192.168.0.0/16#' custom-resources.yaml
grep -n cidr custom-resources.yaml
```

Run the grep again. It must now show your Step 6 range. (If you chose a different pod range in Step 6, put that range in place of 192.168.0.0/16 in the sed command.) On VirtualBox with 10.244.0.0/16 the first grep already matches and no sed is needed.

Fallback if the curl download fails: create the file yourself.

```
cat > custom-resources.yaml <<EOF
apiVersion: operator.tigera.io/v1
kind: Installation
metadata:
  name: default
spec:
  calicoNetwork:
    ipPools:
    - name: default-ipv4-ippool
      blockSize: 26
      cidr: 192.168.0.0/16
      encapsulation: VXLANCrossSubnet
      natOutgoing: Enabled
      nodeSelector: all()
---
apiVersion: operator.tigera.io/v1
kind: APIServer
metadata:
  name: default
spec: {}
EOF
```

8c. Install Calico and wait:

```
kubectl create -f custom-resources.yaml
```

Then, as a separate command:

```
watch kubectl get pods -n calico-system
```

Press Ctrl+C to stop watching. All pods must reach Running (this takes a few minutes). Then:

```
kubectl get tigerastatus
kubectl get nodes
```

tigerastatus shows Available for each component, and the master turns Ready.

If calico-system is still empty after 3 to 4 minutes, run kubectl get tigerastatus. If the message mentions an IPPool CIDR that is "not within the platform's configured pod network CIDR", go to Problem 14. For any other cause, see Problem 6.

8d. VirtualBox only: make Calico use the Host-only interface. With two adapters, Calico can pick the NAT interface (enp0s3) for node-to-node traffic, which leaves workers NotReady or pods unable to talk across nodes (Problem 25). After 8c, tell the operator to use enp0s8 (confirm the name with ip a):

```
kubectl patch installation default --type=merge -p '{"spec":{"calicoNetwork":
{"nodeAddressAutodetectionV4":{"interface":"enp0s8"}}}}'
```

Then check kubectl get pods -n calico-system -o wide again.

Part C: Join the workers (each worker)

Only workers join. The master never runs kubeadm join, and you do not install Calico on workers.

Step 9. Get the join command (on the master)

This prints a complete, single-line join command. Tokens last 24 hours, so run it again whenever you need a new one:

```
kubeadm token create --print-join-command
```

The address in it must be the master's private IP (kubeadm join 192.168.56.10:6443 on VirtualBox). If it shows 10.0.2.15, the master was initialized with the NAT address and the workers can never join (Problems 17 and 18).

Step 10. Run the join command on each worker

Before joining, make sure the worker passed the Part A final check, uses the same Kubernetes version as the master, and can reach the master:

```
ping -c 3 192.168.56.10
nc -zv 192.168.56.10 6443
```

If the worker was used in an earlier attempt, reset it first and set its own node IP (VirtualBox: .11 on worker1, .12 on worker2):

```
sudo swapoff -a
sudo kubeadm reset -f
echo 'KUBELET_EXTRA_ARGS=--node-ip=192.168.56.11' | sudo tee
/etc/default/kubelet
```

Then copy the entire line the master printed and put sudo in front of it. It looks like this:

```
sudo kubeadm join <MASTER_PRIVATE_IP>:6443 --token <token> --discovery-token-
ca-cert-hash sha256:<hash>
```

Joining takes about a minute. If it hangs or retries, see Problem 7 (cannot connect), Problem 17 (wrong IP) and Problem 24 (TLS handshake timeout).

Part D: Verify the cluster (master)

Step 11. Check nodes and system pods

```
kubectl get nodes -o wide
kubectl get pods -A
```

All three nodes must show Ready, and the pods must be Running (give it a few minutes after the last join). A worker that has just joined shows NotReady until its calico-node pod is running, which takes 3 to 5 minutes while images download. Watch it with:

```
kubectl get pods -n calico-system -o wide
```

On VirtualBox the INTERNAL-IP column must show 192.168.56.10, .11 and .12, not 10.0.2.15. If a worker stays NotReady, see Problem 25. Optionally label the workers:

```
kubectl label node k8s-worker1 node-role.kubernetes.io/worker=
kubectl label node k8s-worker2 node-role.kubernetes.io/worker=
```

Step 12. Deploy a test app

```
kubectl create deployment nginx --image=nginx --replicas=2
kubectl expose deployment nginx --type=NodePort --port=80
kubectl get pods -o wide
kubectl get svc nginx
```

The service shows a port such as 80:31234/TCP. Open http://<worker-ip>:31234 in a browser (use your own port; on AWS use the worker's public IP and allow the port in the security group; on VirtualBox use the worker's Host-only IP from the Windows host). The nginx welcome page means your cluster works. When you are done testing, clean up:

```
kubectl delete service nginx
kubectl delete deployment nginx
```

Final success checklist

- kubectl get nodes shows all three nodes Ready
- kubectl get pods -A shows everything Running or Completed
- kubectl get tigerastatus shows Calico components Available
- The nginx test page opens in the browser
- After a reboot of the VMs, free -h still shows 0B swap and kubectl get nodes still works

8. On-Prem Extras You Must Plan

Load balancer for Services (MetalLB)

On-prem there is no cloud load balancer, so type: LoadBalancer Services stay Pending. MetalLB hands out IPs from a range you reserve on your network. (Not usable on AWS, see Section 5.1.)

```
kubectl apply -f
https://raw.githubusercontent.com/metallb/metallb/v0.14.9/config/manifests/me
tallb-native.yaml
```

Create metallb-config.yaml with a free IP range from your LAN, then apply it:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata:
  name: lan-pool
  namespace: metallb-system
spec:
  addresses:
  - 192.168.1.200-192.168.1.220
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata:
  name: lan-adv
  namespace: metallb-system
```

```
kubectl apply -f metallb-config.yaml
```

Persistent storage

Pods lose local data when they restart, so use shared storage for databases and uploads. Common on-prem choices:

- NFS with an NFS provisioner: simplest, good for labs and small setups
- Longhorn: easy replicated storage using the Nodes' own disks
- Ceph (Rook): powerful and scalable, but more complex

High availability for production

One master is a single point of failure. For production use 3 control-plane nodes behind a virtual IP provided by HAProxy plus keepalived, and initialize with that address:

```
sudo kubeadm init --control-plane-endpoint=192.168.1.100:6443 --upload-certs
--pod-network-cidr=192.168.0.0/16
```

Back up etcd regularly; it holds the whole cluster state.

Firewall ports to open between nodes

Node type	Port	Used for
Control plane	6443 TCP	API Server (kubectl and nodes)
Control plane	2379-2380 TCP	etcd
Control plane	10257, 10259 TCP	Controller Manager, Scheduler
All nodes	10250 TCP	Kubelet API
All nodes	4789 UDP	Calico VXLAN (Pod traffic between nodes)
Workers	10256 TCP	kube-proxy
Workers	30000-32767 TCP	NodePort Services

On AWS, the easy way to cover all the node-to-node rows is one security group rule: allow all traffic from the same security group. On a VirtualBox lab, run sudo ufw status on every node; if it is active, either open these ports or run sudo ufw disable (lab only).

9. Common Problems and Fixes

Quick table

Symptom	Likely cause	Fix
Node stays NotReady	No Pod network installed, or Calico pods not running or still pulling	Complete Step 8 and check kubectl get pods -n calico-system -o wide (Problem 25)
No resources found in calico-system namespace and tigerastatus mentions an IPPool CIDR	CIDR in custom-resources.yaml differs from kubeadm init	See Problem 14
kubeadm init fails on preflight	Swap on, or required ports in use	Repeat Step 2, run sudo kubeadm reset -f and try again

Symptom	Likely cause	Fix
Kubelet keeps restarting after init/join or after a reboot	Swap is on, or containerd not using systemd cgroups	Turn swap off (Problem 16), or repeat Step 4 and run sudo systemctl restart containerd kubelet
Join command says token invalid	Token older than 24 hours	Run kubeadm token create --print-join-command on the master
Service stays Pending on LoadBalancer	No load balancer on-prem	Install MetalLB (on AWS use the AWS Load Balancer Controller)
ip_forward prints 0 or Path is not under /proc/sys/	sysctl file damaged when pasted	See Problem 13
kubectl says localhost:8080 ... connection refused	No kubeconfig for your user (Step 7 not done)	See Problem 8
kubectl or join says connection refused to 10.0.2.15:6443	VirtualBox NAT IP used, or kubelet down	Problems 15, 16, 17 and 18
Join retries with TLS handshake timeout	API server busy or not ready	Problem 24
vim: command not found, or terminal stuck on (END)	Minimal Ubuntu, or pager	Problems 21 and 22

Problems hit while building this cluster

Problem 1: E: Unable to locate package kubelet and Unit kubelet.service does not exist

What you see:

```
E: Unable to locate package kubelet
E: Unable to locate package kubeadm
E: Unable to locate package kubectl
E: No packages found
Failed to enable unit: Unit kubelet.service does not exist
```

Cause: the Kubernetes apt repository was never added. The Hit: lines show only ubuntu.com URLs and no pkgs.k8s.io.

Fix: run Step 5 (5a to 5d). If the repository is added but you now see NO_PUBKEY, go to Problem 9.

Problem 2: /etc/hosts contains public IPs or duplicate lines

What you see: entries such as 16.4.72.94 k8s-master (AWS public addresses), or the same hostname listed twice because tee -a was run more than once.

Cause: public IPs were used instead of private IPs, or the append command was repeated. Public IPs change on stop/start, are slower, and are not what the API server advertises.

Fix: find the private IPs (hostname -I on AWS, ip a show enp0s8 on VirtualBox), then edit /etc/hosts on all nodes and keep exactly one line per node:

```
172.31.11.184 k8s-master
<worker1-private-ip> k8s-worker1
<worker2-private-ip> k8s-worker2
```

Problem 3: kubeadm init fails at wait-control-plane with context deadline exceeded

What you see:

```
error execution phase wait-control-plane: cannot obtain client without
bootstrap:
could not bootstrap the admin user in file admin.conf: unable to create
ClusterRoleBinding:
Post
"https://192.168.1.10:6443/apis/rbac.authorization.k8s.io/v1/clusterrolebindi
ngs?timeout=10s":
context deadline exceeded
```

Cause: the guide's example IP 192.168.1.10 did not exist on the machine. hostname -I showed the real IP was 172.31.11.184. The API server was told to advertise an address nobody could reach, so the init timed out. (Certificates and manifests are created fine before this point.)

Fix:

```
hostname -I
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d $HOME/.kube
sudo systemctl restart containerd kubelet
MASTER_IP=$(hostname -I | awk '{print $1}')
sudo kubeadm init --apiserver-advertise-address=$MASTER_IP --pod-network-
cidr=192.168.0.0/16
```

On VirtualBox set MASTER_IP=192.168.56.10 by hand instead (Problem 18). Then repeat Step 7. If it still fails, check:

- grep SystemdCgroup /etc/containerd/config.toml shows true, and containerd was restarted
- The instance has at least 2 vCPU and about 2 GB RAM (use t3.medium or larger on AWS)
- cat /proc/sys/net/ipv4/ip_forward prints 1 (if not, repeat Step 3)
- Swap is off (free -h)
- The logs show the real error:

```
sudo journalctl -u kubelet --no-pager | tail -30
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps -a
```

Problem 4: Pod CIDR overlaps your network

Cause: the pod range overlaps the VPC, LAN or Host-only range. If your network already uses 192.168.x.x (the VirtualBox Host-only network is 192.168.56.0/24), the pod range 192.168.0.0/16 collides with it and routing breaks. (For a mismatch between kubeadm init and Calico's file, see Problem 14.)

Fix: pick one pod range that does not overlap your network and use it in both places. The default AWS VPC (172.31.0.0/16) does not overlap with 192.168.0.0/16. If your network uses 192.168.x.x, choose another range such as 10.244.0.0/16, use it in Step 6, and set the same value in custom-resources.yaml (Step 8b).

Problem 5: Calico install command errors (old URLs)

Cause: older guides use v3.29.3, with tigera-operator.yaml and custom-resources.yaml only. Newer Calico releases ship the CRDs as a separate manifest.

Fix: use the commands in Step 8 (v3.32.2) and use kubectl create, not apply.

Problem 6: No resources found in calico-system namespace

What you see: watch kubectl get pods -n calico-system shows nothing after the install commands.

Cause: one of these:

- It is only timing (wait 1 to 2 minutes).
- An earlier step did not complete: the operator is not running, or custom-resources.yaml was never applied.
- The CIDR in custom-resources.yaml does not match kubeadm init. This is the most common cause when waiting does not help. See Problem 14.

Fix, check in this order:

```
kubectl get pods -n tigera-operator
kubectl get tigerastatus
kubectl logs -n tigera-operator deploy/tigera-operator --tail=40
kubectl get installation
grep -n cidr custom-resources.yaml
```

If tigerastatus shows an IPPool / CIDR message, go to Problem 14. If kubectl create -f custom-resources.yaml says the resource already exists, it was created earlier, so skip it and wait. Also scroll up and check that each earlier kubectl create printed created and not an error. Remove any leftover Flannel install (see Step 8).

Problem 7: kubeadm join hangs or times out

Cause: the AWS security group (or firewall) blocks traffic between nodes, the VMs are not on the same network, or the join address is wrong.

Fix:

- Join using the master's private IP on port 6443 (on VirtualBox the Host-only IP, never 10.0.2.15)
- Allow all traffic from the same security group (or open the ports in the Section 8 table)
- On VirtualBox confirm all VMs use the same Host-only network and that pings succeed
- From the worker, test the connection:

```
nc -vz <MASTER_PRIVATE_IP> 6443
```

Problem 8: The connection to the server localhost:8080 was refused

What you see (usually right after kubeadm init succeeded):

```
E1023 07:15:58.193144   10982 memcache.go:265] "Unhandled Error"
err="couldn't get current server API group list: Get
\"http://localhost:8080/api?timeout=32s\": dial tcp 127.0.0.1:8080: connect:
connection refused"
The connection to the server localhost:8080 was refused - did you specify the
right host or port?
```

Cause: kubectl has no kubeconfig for your user, so it falls back to localhost:8080, where nothing is listening. The cluster itself is fine. kubeadm init only creates the config at /etc/kubernetes/admin.conf (readable by root). It does not copy it for your user, so Step 7 is required.

Fix: run Step 7 on the master:

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
kubectl get nodes
```

kubectl get nodes must now list k8s-master (status NotReady until Calico is installed). Notes:

- Run these as your normal user (for example ubuntu), not as root with sudo su. The $HOME/.kube/config file must belong to the user who runs kubectl.
- If cp asks overwrite?, answer y.
- Kubectl only works on the master (where admin.conf exists). Workers do not need it.
- Quick temporary alternative for the current session only: export KUBECONFIG=/etc/kubernetes/admin.conf (you may need sudo kubectl … with it). The permanent fix above is better.

Problem 9: NO_PUBKEY / "The repository … is not signed" and packages still not found

What you see:

```
Err:4 https://pkgs.k8s.io/core:/stable:/v1.34/deb  InRelease
The following signatures couldn't be verified because the public key is not
available: NO_PUBKEY 234654DA9A296436
E: The repository 'https://pkgs.k8s.io/core:/stable:/v1.34/deb  InRelease' is
not signed.
E: Unable to locate package kubelet
```

Cause: the repository line exists, but the signing key file is missing or empty. This happens when the long curl ... | sudo gpg --dearmor -o ... command gets a line break while copying (for example from a PDF), so the key file is never created. It can also happen if the old key file existed and gpg could not overwrite it.

Fix: redo the key in separate short commands on the failing node. Use the same version as the other nodes (shown here as v1.35; use v1.34 if that is what your cluster uses):

```
KUBE_VERSION=v1.35
KEYRING=/etc/apt/keyrings/kubernetes-apt-keyring.gpg
REPO=https://pkgs.k8s.io/core:/stable:/$KUBE_VERSION/deb
sudo mkdir -p -m 755 /etc/apt/keyrings
sudo rm -f $KEYRING
curl -fsSL $REPO/Release.key -o /tmp/k8s-release.key
sudo gpg --dearmor -o $KEYRING /tmp/k8s-release.key
sudo chmod 644 $KEYRING
ls -l $KEYRING
```

The ls line must show a size greater than 0. Then make sure the repo file is one clean line:

```
echo "deb [signed-by=$KEYRING] $REPO/ /" | sudo tee
/etc/apt/sources.list.d/kubernetes.list
cat /etc/apt/sources.list.d/kubernetes.list
```

The cat output must be a single line with the real version number and no $ or ${...}. Then install:

```
sudo apt-get update
sudo apt-get install -y kubelet kubeadm kubectl
sudo apt-mark hold kubelet kubeadm kubectl
sudo systemctl enable --now kubelet
```

During apt-get update the pkgs.k8s.io line must show Get: or Hit: and no Err:.

Problem 10: Node still has the default hostname (for example ip-172-31-8-191)

Cause: the hostname was never set, or two machines share the same name (common with cloned VMs). Names must be unique and should match /etc/hosts.

Fix: on each node, before kubeadm init or kubeadm join:

```
sudo hostnamectl set-hostname k8s-worker1
hostname
```

Use k8s-master, k8s-worker1 or k8s-worker2 as appropriate. If the node already joined with the wrong name, reset it (Section 10), rename it, and join again.

Problem 11: Mixed Kubernetes versions across nodes

Cause: some nodes were installed with one minor version (for example v1.34) and others with another (for example v1.35) because the version variable differed.

Fix: check on every node with kubeadm version -o short. Reinstall the odd node using the version of the rest (Section 10, "Change the Kubernetes version on a node"), or upgrade the whole cluster properly.

Problem 12: Commands break when pasted

Cause: copying long commands from a PDF or a wrapped screenshot can insert line breaks. Pasting several commands at once can also glue them together on one line (see Problem 23).

Fix: copy from the .md file or GitHub, copy one block at a time, and prefer one-line commands (this guide uses printf ... | sudo tee for most config files). If something looks cut off, run history | tail to see exactly what the shell received.

Problem 13: ip_forward prints 0 and sysctl: Path is not under /proc/sys/

What you see:

```
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
sysctl: Path is not under /proc/sys/: /proc/sys/
```

and cat /proc/sys/net/ipv4/ip_forward prints 0.

Cause: the third line of /etc/sysctl.d/k8s.conf (net.ipv4.ip_forward = 1) was damaged when the multi-line command was pasted, so sysctl stopped before applying it. The two bridge settings were applied, but ip_forward was not. Without ip_forward = 1, kubeadm init and kubeadm join fail their preflight checks.

Fix: look at the file, then rewrite it with a one-line command and apply it:

```
cat -A /etc/sysctl.d/k8s.conf
printf '%s\n' 'net.bridge.bridge-nf-call-iptables = 1' 'net.bridge.bridge-nf-
call-ip6tables = 1' 'net.ipv4.ip_forward = 1' | sudo tee
/etc/sysctl.d/k8s.conf
sudo sysctl --system
cat /proc/sys/net/ipv4/ip_forward
```

The last command must print 1, and sysctl --system must not show the "Path is not under" error. If the error still appears, another file causes it. Find it with:

```
sudo grep -rn "/proc/sys" /etc/sysctl.d /etc/sysctl.conf
```

Quick temporary workaround (lost after a reboot, so still do the fix above):

```
sudo sysctl -w net.ipv4.ip_forward=1
```

Check every node, because the same paste problem can hit any of them:

```
cat /proc/sys/net/ipv4/ip_forward
```

Problem 14: Calico never starts: IPPool CIDR is "not within the platform's configured pod network CIDR"

What you see: after Step 8c, nothing appears in calico-system, even after 10 minutes or more. The master stays NotReady and both coredns pods stay Pending:

```
kubectl get pods -n calico-system
No resources found in calico-system namespace.
```

The operator pod is Running and kubectl get installation shows default, so it looks like everything is installed. The real error is in tigerastatus:

```
kubectl get tigerastatus
NAME        AVAILABLE   PROGRESSING   DEGRADED   MESSAGE
apiserver               True          Error creating / updating resource:
namespaces "calico-system" not found
calico                  True          waiting for enabled IP pools to be
created
ippools                 True          error filling IP pool defaults: could
not resolve CalicoNetwork IPPool
                                      and kubeadm configuration: IPPool
10.244.0.0/16 is not within the
                                      platform's configured pod network
CIDR(s) [192.168.0.0/16]
```

Cause: the pod CIDR is set in two places and they do not match:

- kubeadm init --pod-network-cidr=192.168.0.0/16 (Step 6) sets the range Kubernetes expects for pods.
- custom-resources.yaml (Step 8b) sets the range Calico tries to create. The file downloaded for Calico v3.32.2 contains cidr: 10.244.0.0/16.

The Tigera operator reads the kubeadm range, sees that Calico's pool is outside it, and refuses to create the IP pool. Because it stops there, it never creates the calico-system namespace, and so no Calico pods start. Waiting does not help, and the operator itself shows no crash. The cause only appears in tigerastatus or the operator logs.

Fix (on the master):

1. Confirm the mismatch. Line 13 normally holds the CIDR:

```
grep -n cidr custom-resources.yaml
```

2. Change the file to the range used in Step 6, then check it:

```
sed -i 's#10.244.0.0/16#192.168.0.0/16#' custom-resources.yaml
grep -n cidr custom-resources.yaml
```

The second command must show cidr: 192.168.0.0/16.

3. Apply the corrected file. Use apply here (not create), because the resources already exist:

```
kubectl apply -f custom-resources.yaml
```

Each of the four resources should print configured. You will also see warnings that the last-applied-configuration annotation is missing. These are harmless: the resources were first made with create, and kubectl patches the annotation automatically.

4. Restart the operator so it retries immediately:

```
kubectl rollout restart deployment tigera-operator -n tigera-operator
```

5. Watch Calico start (as its own command). This takes 2 to 5 minutes:

```
watch kubectl get pods -n calico-system
```

Press Ctrl+C when calico-node, calico-typha and calico-kube-controllers are Running.

6. Confirm the cluster is healthy:

```
kubectl get tigerastatus
kubectl get nodes
kubectl get pods -n kube-system
```

Every component must show Available, the master must be Ready, and the coredns pods must change from Pending to Running.

If the tigerastatus message still mentions the old CIDR after step 5, the existing Installation kept the old value. Delete and recreate it. This is safe on a fresh cluster, because no workloads depend on Calico yet:

```
kubectl delete -f custom-resources.yaml
kubectl create -f custom-resources.yaml
kubectl rollout restart deployment tigera-operator -n tigera-operator
```

Related mistake: watch pasted onto the create line. If you paste these two lines together, the shell runs them as one command and prints watch kubectl get pods -n calico-system as extra arguments. The create may still succeed, but the watch never runs, and you may see Stopped watch kubectl ... job messages. Ignore the stopped jobs, and run the two commands one at a time.

Prevention: before you run kubectl create -f custom-resources.yaml (Step 8c), always run grep -n cidr custom-resources.yaml and compare it with the --pod-network-cidr you used in Step 6. Both must be identical.

Problems hit on VirtualBox (Revision 4)

Problem 15: All VMs show the same IP (10.0.2.15)

What you see: ip a on the master, worker1 and worker2 all show inet 10.0.2.15/24 on enp0s3.

Cause: VirtualBox NAT mode. Each VM sits on its own private NAT network, so the VMs cannot reach each other and Kubernetes cannot work.

Fix: add a second Host-only adapter and give each VM a static IP (Section 5.2). Use only the Host-only IPs in /etc/hosts, kubeadm and kubelet.

Problem 16: Kubelet keeps restarting and kubectl says connection refused after a reboot

What you see:

```
The connection to the server 10.0.2.15:6443 was refused - did you specify the
right host or port?
sudo systemctl status kubelet  ->  activating (auto-restart) (Result: exit-
code), status=1/FAILURE
```

The kubelet log (sudo journalctl -u kubelet -n 30 --no-pager) shows:

```
running with swap on is not supported, please disable swap or set --fail-
swap-on flag to false
Swap is on  /proc/swaps contents: /swapfile  file  2744316
```

Cause: swap came back after a reboot. Ubuntu Desktop uses /swapfile, which is re-enabled from /etc/fstab at every boot. Kubelet crashes, so the API server never starts and nothing listens on port 6443.

Fix (every node):

```
sudo swapoff -a
sudo sed -i '/swap/ s/^/#/' /etc/fstab
sudo systemctl restart kubelet
free -h
grep swap /etc/fstab
```

The Swap row must show 0B and the fstab line must start with #. Wait about a minute, then run kubectl get nodes. Do this on all three nodes before any init or join.

Problem 17: kubeadm join retries forever against 10.0.2.15:6443

What you see on the worker:

```
Failed to request the cluster-info ConfigMap: Get
"https://10.0.2.15:6443/api/v1/namespaces/kube-public/configmaps/cluster-
info?timeout=10s": dial tcp 10.0.2.15:6443: connect: connection refused
```

Cause: the master was initialized while only the NAT IP was in use, so the join command points to 10.0.2.15. On the worker, 10.0.2.15 is the worker itself, so the connection is refused. A new token does not help.

Fix: reset and re-init the master with the Host-only IP (Problem 18), then generate a new join command. It must start with kubeadm join 192.168.56.10:6443. Reset each worker first with sudo kubeadm reset -f.

Problem 18: Cluster was initialized with the wrong IP (NAT address)

Cause: MASTER_IP=$(hostname -I | awk '{print $1}') returns 10.0.2.15 on VirtualBox because the NAT address is listed first. Certificates, kubeconfig and the advertised address were then created for 10.0.2.15. Changing the IP afterwards does not fix an existing cluster.

Fix (master):

```
sudo swapoff -a
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d $HOME/.kube
sudo nmcli connection delete vxlan.calico
sudo ip link delete vxlan.calico
echo 'KUBELET_EXTRA_ARGS=--node-ip=192.168.56.10' | sudo tee
/etc/default/kubelet
sudo systemctl restart containerd kubelet
MASTER_IP=192.168.56.10
sudo kubeadm init --apiserver-advertise-address=$MASTER_IP --pod-network-
cidr=10.244.0.0/16
```

The two vxlan.calico commands print an error if the interface does not exist; that is fine. Then repeat Step 7, install Calico again (Step 8) and join the workers with the new command (Step 9 and Step 10).

Problem 19: Leftover vxlan.calico after kubeadm reset

What you see: nmcli connection show still lists vxlan.calico (type vxlan) after a reset.

Cause: kubeadm reset does not remove network interfaces created by Calico.

Fix (on every node that ran Calico):

```
sudo nmcli connection delete vxlan.calico
sudo ip link delete vxlan.calico
```

If network problems continue after a reset, reboot the node (see Section 10 for flushing iptables).

Problem 20: Host-only IP is dynamic, or the netplan file has no effect

What you see: ip a shows 192.168.56.102/24 ... scope global dynamic on enp0s8 with a lease of a few hundred seconds. ls /etc/netplan/ shows only 01-network-manager-all.yaml.

Cause: enp0s8 uses DHCP, so the address can change, and Kubernetes needs a fixed IP. On Ubuntu Desktop, netplan only hands control to NetworkManager, so editing the netplan file does not set a static IP.

Fix: set the static IP on the NetworkManager connection (Section 5.2.3):

```
nmcli -p connection show | cat
sudo nmcli connection modify "Wired connection 2" ipv4.method manual
ipv4.addresses 192.168.56.10/24
sudo nmcli connection up "Wired connection 2"
ip a show enp0s8
```

The address must no longer say dynamic. Use .11 and .12 on the workers.

Problem 21: sudo: vim: command not found

Cause: a minimal Ubuntu install has only nano and a basic vi.

Fix: use nano, or install vim:

```
sudo nano hello.txt
sudo apt update
sudo apt install -y vim
```

In nano save with Ctrl+O then Enter, exit with Ctrl+X. You also do not need sudo to create files in your home directory; sudo makes them owned by root.

Problem 22: Terminal stuck on (END) or lines 1-8/8

Cause: systemctl status, journalctl and nmcli open their output in a pager.

Fix: press q. To avoid the pager, add --no-pager or pipe to cat:

```
sudo systemctl status kubelet --no-pager
nmcli -p connection show | cat
sudo journalctl -u kubelet -n 30 --no-pager
```

Problem 23: Several pasted commands ran as one

Cause: three commands (systemctl status kubelet, systemctl status containerd, crictl ps) were pasted together and ended up as one command line, so only the first ran and a sudo password prompt appeared in the middle.

Fix: paste one command at a time, and check the output before the next one. Use a backslash only to continue a single long command.

Problem 24: kubeadm join fails with TLS handshake timeout or context deadline exceeded

What you see on the worker, now pointing at the correct IP:

```
Get "https://192.168.56.10:6443/api/v1/namespaces/kube-
public/configmaps/cluster-info?timeout=10s": net/http: TLS handshake timeout
```

Meaning: the worker reaches the master, but the API server does not answer in time. This is progress compared with Problem 17: the IP is now correct. It usually clears when the control plane has finished starting.

Check on the master:

```
kubectl get nodes
kubectl get pods -n kube-system
sudo ss -tlnp | grep 6443
sudo systemctl status kubelet --no-pager
sudo crictl ps -a | grep -E 'apiserver|etcd'
free -h
```

Check on the worker:

```
ping -c 3 192.168.56.10
nc -zv 192.168.56.10 6443
curl -k --max-time 10 https://192.168.56.10:6443/healthz
```

- curl returns ok: the API server is fine, run the join again.

- ping works but curl times out: the API server is overloaded or restarting. Wait until every kube-system pod is Running (2 to 3 minutes after init or after the Calico install) and check the master has enough RAM and CPU.
- ping fails: the two VMs are not on the same Host-only network (Section 5.2.2).

Also check sudo ufw status (run sudo ufw disable in a lab) and that date is similar on all nodes. Then retry:

```
sudo kubeadm reset -f
```

On the master create a fresh command with kubeadm token create --print-join-command, and run it on the worker with sudo.

Problem 25: Workers joined but stay NotReady

What you see:

```
kubectl get nodes
k8s-master    Ready      control-plane   8m23s   v1.35.10
k8s-worker1   NotReady   <none>          3m15s   v1.35.10
k8s-worker2   NotReady   <none>          3m25s   v1.35.10
```

Meaning: the join worked, which proves the IP and network fixes are right. A node stays NotReady until the calico-node pod on it is running. Image pulls over the NAT connection can take 3 to 5 minutes, so first just wait.

Check on the master:

```
kubectl get pods -n calico-system -o wide
kubectl get nodes -o wide
kubectl describe node k8s-worker1 | grep -A8 Conditions
```

calico-node must show 1/1 Running on all three nodes. Look for Init, ContainerCreating or ImagePullBackOff and read the Events at the bottom of kubectl describe pod <name> -n calico-system.

If it is still NotReady after about 5 minutes, check these on the worker and fix what is wrong:

- Swap on again: free -h must show 0B (Problem 16). Then sudo systemctl restart kubelet.
- Wrong node IP: INTERNAL-IP must be 192.168.56.11 or .12, not 10.0.2.15. Set --node-ip in /etc/default/kubelet and restart kubelet.

- Calico using the NAT interface: apply Step 8d (nodeAddressAutodetectionV4 interface enp0s8) on the master.
- Firewall: run sudo ufw status, and open 4789/udp and 10250/tcp or disable ufw in a lab.
- Kubelet errors: sudo journalctl -u kubelet -n 30 --no-pager, and sudo crictl images | grep calico to see if images arrived.

Useful debugging commands

```
sudo journalctl -u kubelet -n 30 --no-pager
sudo journalctl -u kubelet -f
kubectl describe pod <name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl get tigerastatus
kubectl logs -n tigera-operator deploy/tigera-operator --tail=40
sudo crictl --runtime-endpoint unix:///run/containerd/containerd.sock ps -a
ip a
curl -k --max-time 10 https://<MASTER_IP>:6443/healthz
```

10. Starting Over (Reset and Uninstall)

Reset a node (keeps the installed packages)

On the node you want to wipe (master or worker):

```
sudo kubeadm reset -f
sudo rm -rf /etc/cni/net.d $HOME/.kube
sudo systemctl restart containerd kubelet
```

On a node that ran Calico (VirtualBox or anywhere), also remove the leftover interface, because kubeadm reset does not (Problem 19):

```
sudo nmcli connection delete vxlan.calico
sudo ip link delete vxlan.calico
```

If that node was a worker that had joined, also remove it from the cluster on the master:

```
kubectl delete node <node-name>
```

kubeadm reset does not clean iptables rules. If you hit odd network problems after a reset, either reboot the node or flush them (this removes all firewall rules on the node, so only do it on a node you are rebuilding):

```
sudo iptables -F
sudo iptables -t nat -F
sudo iptables -t mangle -F
sudo iptables -X
```

After any reboot, check swap before anything else: free -h must show 0B.

Change the Kubernetes version on a node

Use this to fix a node installed with the wrong version, then rerun Step 5 with the right KUBE_VERSION:

```
sudo kubeadm reset -f
sudo apt-mark unhold kubelet kubeadm kubectl
sudo apt-get purge -y kubelet kubeadm kubectl
sudo rm -f /etc/apt/sources.list.d/kubernetes.list
sudo rm -f /etc/apt/keyrings/kubernetes-apt-keyring.gpg
```

Rebuild the whole cluster

1. Reset every node (first block above), and confirm swap is off.
2. Check each node's IP is the right one (on VirtualBox 192.168.56.10, .11, .12) and all pings work.
3. On the master, repeat Part B (Steps 6 to 8). Check the CIDR in Step 8b again.
4. Generate a new join command (Step 9), confirm it starts with the master's private IP, and join the workers (Step 10).

Quick order to build a VirtualBox cluster from scratch

1. Each VM: Adapter 2 Host-only, static IP with nmcli, all pings work (Section 5.2).
2. Each VM: hostname, /etc/hosts with 192.168.56.x, swap off permanently, kernel modules, containerd, kubeadm, kubelet --node-ip (Part A).
3. Master: MASTER_IP=192.168.56.10, kubeadm init with 10.244.0.0/16, kubeconfig, Calico, Step 8d (Part B).

4. Master: kubeadm token create --print-join-command. Workers: run it with sudo (Part C).
5. Master: kubectl get nodes -o wide and kubectl get pods -A. All Ready and Running (Part D).

11. Command Cheat Sheet

Task	Command
List nodes	kubectl get nodes -o wide
List all pods	kubectl get pods -A
Describe a resource	kubectl describe pod <name>
See logs	kubectl logs <pod-name>
Apply a YAML file	kubectl apply -f file.yaml
Scale an app	kubectl scale deployment nginx --replicas=4
Delete an app	kubectl delete deployment nginx
New join command	kubeadm token create --print-join-command
Check installed version	kubeadm version -o short
Find the node's private IP	hostname -I (AWS, LAN) or ip a show enp0s8 (VirtualBox)
Check Calico health	kubectl get tigerastatus
Check Calico pods per node	kubectl get pods -n calico-system -o wide
Check pod CIDR in Calico file	grep -n cidr custom-resources.yaml
Reset a node	sudo kubeadm reset -f
Check swap is off	free -h and grep swap /etc/fstab
Turn swap off permanently	sudo swapoff -a and sudo sed -i '/swap/ s/^/#/' /etc/fstab
Kubelet error reason	sudo journalctl -u kubelet -n 30 --no-pager
List NetworkManager connections	nmcli -p connection show | cat

Task	Command
Set a static IP	sudo nmcli connection modify "Wired connection 2" ipv4.method manual ipv4.addresses 192.168.56.10/24
Remove leftover Calico interface	sudo nmcli connection delete vxlan.calico and sudo ip link delete vxlan.calico
API server health from a worker	curl -k --max-time 10 https://192.168.56.10:6443/healthz
Test a port from a worker	nc -zv 192.168.56.10 6443